INDEX

Topic	Page No
Conditional Statements	3
While loop	6
For loop	6
Break &Continue Statements	7
Pass	8
Star Programs	9
Number patterens	11
Alphabet Patterens	14
List	17
Tuple	18
Set	20
Dictionary	21
List Comprehension	23
Set Comprehension	23
Dictionary Comprehension	24
Shuffling	25
Swapping	25
Sorting	26
Strings	27
String Methods	28
Sliceing	31
Functions	32
Lambda Functions	33
Recursion Functions	34
Functions Arguments	34

Decorators 36

Closures 36

Generator 37

Class &Objects 37

Inheritance 38

Abstraction 39

Polymorphism 40

Method-Overriding 42

Static-Methods 43

Property Decorator 44

Encapsulation 44

CONDITIONAL STATEMENTS

If Statement: 13/07/2026

1. Write a Python program that checks if a number is positive.
2. Write a Python program that checks if a given string is empty.
3. Write a Python program that determines if a number is positive, negative, or zero using only if statements and no elif or else.
4. Write a Python program that checks if a number is a multiple of both 3 and 5.
5. Write a Python program that determines if a given number is a perfect square.
6. Write a Python program that checks if a number is divisible by both 2 and 3.
7. Write a Python program that determines if a number is a perfect cube.
8. Write a Python program that checks if a given number is a multiple of 4.

If-Else Statement:

1. Write a Python program that checks if a given year is a leap year.
2. Write a Python program that checks if a given character is a vowel or a consonant.
3. Write a Python program that checks if a given year is a century year (ending in '00').
4. Write a Python program that checks if a given number is prime.
5. Write a Python program that checks if a person is eligible to vote based on their age.
6. Write a Python program that checks if a number is positive or non-positive (including zero).
7. Write a Python program that compares two numbers and prints the largest one.
8. Write a Python program that checks if a given character is a letter or not.
9. Write a Python program that finds and prints the smallest of three numbers without using inbuilt methods.
10. Write a Python program to check if a given string is a palindrome.
11. Write a Python program that finds and prints the largest of four numbers.
12. Write a Python program that takes two numbers as input and determines the sign of their difference.
13. Write a Python program that checks if a given year is a leap year using an alternative method.
14. Write a Python program that checks if a number is a multiple of 7 or not.
15. Write a Python program that calculates and prints the absolute value of a given number.

If-Elif-Else Statement:

1. Write a Python program that converts a temperature in Celsius to Fahrenheit or vice versa based on user input.
2. Write a simple calculator program that performs basic arithmetic operations (addition, subtraction, multiplication, division) based on user input.
3. Write a Python program for a simple "Guess the Number" game. Generate a random number, and have the user guess it.
4. Write a Python program that checks the strength of a user's password based on certain criteria.
5. Write a Python program that calculates the final price of an item after applying a discount based on the purchase amount.
6. Write a Python program that simulates an ATM machine, allowing users to check their balance and withdraw money.
7. Write a Python program that greets the user differently based on the time of day.

8. Write a Python program that calculates the final price of an item after applying a discount coupon code.
9. Write a Python program that calculates and prints the grade based on a given score, but also validates the input score.
10. Write a Python program that acts as a simple calculator and allows the user to choose the operation (addition, subtraction, multiplication, division).
11. Write a Python program that determines the number of days in a given month.
12. Write a Python program that calculates a person's Body Mass Index (BMI) and categorizes it into different categories (underweight, normal weight, overweight, obese).
13. Write a Python program that converts letter grades (A, B, C, D, F) to their equivalent GPA values.

Nested If Statement:

1. Given x = 10 and y = 5, write a Python program to determine if x is greater than y. If x is greater than y,check if it's also greater than 15 and print appropriate messages accordingly.
2. Write a Python program that takes a grade as input and prints the corresponding letter grade. If the grade is less than 70, also print "You failed."
3. Write a Python program to check if a person is eligible for a credit card based on their age and income.If the person is underage, print "You are underage." If they are of age but have an income below $30,000, print "Your income is too low for a credit card."
4. Write a Python program to determine if a given number num is even or odd. If it's even, check if it's greater than 10 and print appropriate messages.
5. Write a Python program that calculates the ticket price for a theme park based on age and height. If a person isunder 12 years old and under 4 feet tall, their ticket price is $10. If they are under 12 but 4 feet tall or taller,their ticket price is $15. For individuals 12 and older who are 4 feet tall or taller, the ticket price is $20.
6. Write a Python program for user authentication. Ask the user to enter their username and password. If both theusername and password are correct, print "Access granted." If the username is correct but the password is incorrect,print "Incorrect password." If the username is incorrect, print "Invalid username."
7. Write a Python program for ordering food. Ask the user to select a type of food (e.g., "Burger" or "Pizza").If they choose "Burger," ask if they want fries. If they choose "Pizza," ask if they want extra cheese. Print their order with the additional items if chosen.
8. Write a Python program that takes an integer as input and checks if it's positive, negative, or zero. If it's positive,check if it's even or odd. Print the appropriate messages.
9. Write a Python program that asks for a user's age and whether they have a driver's license. If the user is 18 or older andhas a driver's license, print "You are eligible to drive." Otherwise, print "You are not eligible to drive."
10. Write a Python program that checks if a person is eligible to vote. Ask for the person's age and whether they are a citizen.If the person is at least 18 years old and a citizen, print "You are eligible to vote." Otherwise, print "You are not eligible to vote."
11. Write a Python program that takes three numbers as input and prints them in ascending order.

12. Write a Python program for a store that calculates discounts based on the purchase amount and membership status. Ask theuser for the purchase amount and whether they have a membership. If the purchase amount is greater than $100 and they havea membership, apply a 20% discount. If the purchase amount is greater than $100 without a membership, apply a 10% discount. Otherwise, no discount is applied.
13. Write a Python program that classifies a character as a vowel, consonant, or neither. Ask the user to enter a single character.If it's a vowel (a, e, i, o, u), print "Vowel." If it's a consonant, print "Consonant." If it's neither (e.g., a digit or special character), print "Neither."
14. Write a Python program that recommends books based on the reader's age and genre preference. Ask the user for their age andfavorite genre ("Mystery," "Fantasy," or "Science Fiction"). Recommend different books based on their input.
15. Write a Python program that calculates the price of movie tickets based on age and time of day. Ask the userfor their age and whether it's a matinee show (before 5 PM). If the person is under 12 or over 65,they get a discount. Apply different pricing rules based on age and showtime.
16. Write a Python program for a delivery service that calculates delivery charges based on distance and order amount.Ask the user for the distance (in miles) and order amount. Apply different charges based on the distance and order amount.
17. Write a Python program that calculates employee bonuses based on performance and years of service. Ask the user for theemployee's performance rating (Excellent, Good, or Average) and years of service. Calculate and print the bonus amount.
18. Write a Python program for a library that calculates late fees for borrowed books. Ask the user for the number ofdays the book is overdue and the type of book (regular or reference). Calculate and print the late fee.
19. Write a Python program that determines whether a student is eligible for a scholarship based on their GPA and extracurricularactivities. Ask for the student's GPA and the number of extracurricular activities they are involved in. Award the scholarshipif GPA is above 3.5 and they are involved in at least 3 activities.
20. Write a Python program that recommends clothing based on the weather temperature and whether it's raining. Ask theuser for the temperature (in degrees Celsius) and whether it's raining. Provide clothing recommendations accordingly.
21. Write a Python program that recommends a movie based on the user's preferences. Ask the user for theirpreferred genre and age group, and provide movie recommendations accordingly.
22. Write a Python program that determines whether a credit card application is approved based on the applicant's credit scoreand income. Ask the user for their credit score and annual income, and provide an approval status.
23. Write a Python program for vehicle registration based on the type of vehicle and emissions. Ask the user for thevehicle type (car or truck) and emissions status (clean or high). Provide the registration fee based on the inputs.

While loop:

1. Basic Counting: Write a while loop that counts from 1 to 10 and prints each number.
2. Create a program that asks the user to enter a number, and then use a while loop to count down from that number to 1, printing each value.
3. Write a while loop that calculates the sum of even numbers from 1 to 100.
4. Create a simple number guessing game using a while loop. Generate a random number and have the user guess it, giving hints like "too high" or "too low" until they guess correctly.
5. Use a while loop to calculate the factorial of a given number.
6. Write a while loop that prints all even numbers between 1 and 50.
7. Create a program that calculates the sum of the digits of a given integer using a while loop.
8. Generate a multiplication table for a given number using a while loop.
9. Write a program to find all the factors of a given number using a while loop.
10. Implement a program to reverse a given number using a while loop.
11. Create a program that prompts the user to enter a password. Keep asking until they enter the correct password.
12. Write a program using a while loop to generate the first n terms of the Fibonacci sequence.
13. Build a program that checks if a given number is prime using a while loop.
14. Create a program that checks if a given word or phrase is a palindrome (reads the same forwards and backwards) using a while loop.
15. Write a program to generate the first n rows of Pascal's Triangle using while loops.
16. Create a program that converts a given integer into words (e.g., 123 -> "one hundred twenty-three") using a while loop.
17. Write a program to reverse a given string using a while loop.
18. Implement a program to find the greatest common divisor of two numbers using a while loop and the Euclidean algorithm.
19. Create a program that converts a binary number (entered as a string) to its decimal equivalent using a while loop.
20. Write a program that prints all prime numbers between 1 and 100 using a while loop
21. Implement a program to calculate the result of raising a number to a given power using a while loop, without using the built-in exponentiation operator.
22. Create a program that calculates and prints the prime factorization of a given number using a while loop.
23. Build a program that searches for a specific word in a text using a while loop. Provide the index of the word's first occurrence
24. Implement matrix multiplication (matrix dot product) using nested while loops.
25. Given an unsorted array of integers, find the length of the longest consecutive elements sequence using a while loop.

For Loop:

1. Print numbers from 1 to 5 using a for loop?
2. Calculate the sum of numbers from 1 to 10 using a for loop ?
3. Print even numbers from 1 to 10 using a for loop? 4.Calculate the factorial of a number using a for loop?
4. Print elements of a list using a for loop ?

5. Find the maximum value in a list using a for loop ?
6. Print characters in a string using a for loop ?
7. Count the number of vowels in a string using a for loop ?
8. Iterate over a dictionary and print key-value pairs using a for loop ?
9. Print a pattern using nested for loops ?
10. Calculate the sum of all even numbers from 1 to 100 using a for loop ?
11. Print the first 10 Fibonacci numbers using a for loop ?
12. Find and print the common elements between two lists using a for loop ?
13. Generate a multiplication table for a given number using a for loop ?
14. Calculate the factorial of a number using a for loop with a range starting from 1 ?
15. Print the reverse of a string using a for loop ?
16. Generate a list of squares for numbers from 1 to 10 using a for loop ?
17. Count the number of words in a sentence using a for loop ?
18. Calculate the sum of digits in a number using a for loop ?
19. Generate a list of prime numbers within a given range using a for loop ?
20. Calculate the power of a number using a for loop ?
21. Reverse a list using a for loop ?
22. Find and print the largest element in a list using a for loop ?
23. Calculate the average of numbers in a list using a for loop ?
24. Print a countdown from a given number using a for loop ?
25. Generate a pattern of numbers using nested for loops ?
26. Calculate the product of elements in a list using a for loop ?
27. Find and print the position (index) of an element in a list using a for loop ?
28. Calculate the sum of all odd numbers from 1 to 100 using a for loop ?
29. Print a triangle of asterisks using a for loop ?
30. Calculate the LCM (Least Common Multiple) of two numbers using a for loop ?
31. Count the number of prime numbers in a range using a for loop ?
32. Reverse words in a sentence using a for loop ?
33. Print a diamond pattern using nested for loops ?
34. Calculate the sum of the first n natural numbers using a for loop ?
35. Print a hollow square pattern using nested for loops ?
36. Calculate the factorial of a number using a for loop with a specified step ?
37. Print a right-angled triangle pattern using a for loop ?
38. Generate a list of multiples of a number within a given range using a for loop?

Break and Continue statements

1. Write a Python program to find and print the first prime number greater than 100. Use a loop and the break statement to exit the loop once you find the prime number.
2. Create a Python program to calculate the sum of all numbers from 1 to 100. However, if the number is divisible by both 3 and 5, skip it using the continue statement.
3. Write a Python program to find and print the factors of a given number (input by the user). Use a loop and the break statement to exit the loop once you've found all factors.
4. Implement a program that calculates the average of a list of numbers, but ignore any negative numbers. Use the continue statement to skip negative numbers in the list.
5. Create a Python program that simulates a basic login system. Ask the user to enter a username and password. If the entered password is incorrect, give the user three

attempts. Use the break statement to exit the loop if the password is correct or the user exceeds the maximum attempts.

6. Write a Python program that finds the first 10 even numbers. If a number is divisible by 3, skip it using the continue statement.
7. Implement a program to calculate the factorial of a number entered by the user. Use a loop and the break statement to exit the loop when the calculation is complete.
8. Write a Python program that generates and prints the first 20 terms of the Fibonacci sequence. However, if a term is greater than 100, stop generating the sequence using the break statement.
9. Create a Python program that asks the user to enter a series of numbers. Calculate and print the average of these numbers. If the user enters a negative number, stop taking input and calculate the average of the numbers entered so far using the break statement.
10. Implement a Python program that counts and prints the number of vowels in a given string. Use a loop to iterate through the characters in the string and the continue statement to skip consonants.
11. Write a Python program to print all the prime numbers between 1 and 100. Use a loop and the break statement to exit the loop when you determine a number is not prime.
12. Create a Python program that simulates a basic menu-driven application. The program should display a menu to the user with options like "Add," "View," and "Exit." Use the break statement to exit the menu loop when the user selects "Exit."
13. Write a Python program that asks the user to input a password. If the password is not at least 8 characters long, keep asking for a valid password using a loop. Use the continue statement to skip further processing until a valid password is provided.
14. Write a Python program to find the sum of all even numbers from 1 to 100. However, if a multiple of 7 is encountered, skip it using the continue statement.
15. Create a Python program that generates and prints the first 10 terms of the geometric progression (GP) with a common ratio of 2. Use a loop and the break statement to stop generating the sequence after the 10th term.
16. Implement a Python program to count and print the number of digits in an integer. Use a loop and the continue statement to skip non-digit characters.
17. Write a Python program to simulate a basic ATM machine. The program should repeatedly ask the user for the amount they want to withdraw until a valid amount is entered. Use the continue statement to retry input if the amount is not valid.
18. Create a Python program to find and print the smallest and largest numbers in a list of integers. Use a loop, break, and continue to accomplish this.
19. Write a Python program that simulates a basic game where the player has to guess a secret number between 1 and 100. The program should give the player up to 5 attempts, and use the break statement to exit the game when the player either guesses the correct number or runs out of attempts.
20. Implement a Python program that prints all the numbers from 1 to 50. However, if a number is a multiple of both 4 and 7, skip it using the continue statement.

Pass:

1. Create a Python program that iterates over a list of integers and prints all even numbers. For odd numbers, use the pass statement to do nothing.

2. Write a Python program that asks the user to input a list of numbers. For each number in the list, check if it's a prime number. If it's prime, print it; if not, use the pass statement.
3. Implement a Python program that simulates a basic class structure for a geometric shape. Create a class named Shape, and for now, use the pass statement to indicate that further methods and attributes need to be added in the future.
4. Write a Python program that reads a text file and counts the number of lines that contain the word "apple." For lines that don't contain the word, use the pass statement.
5. Create a Python program to check if a given number is a perfect square. If it is, print a message; otherwise, use the pass statement.
6. Implement a Python program that iterates through a list of tasks. For each task, check if it's completed. If it's completed, print a message; if not, use the pass statement.
7. Create a Python program that defines a class called Employee. Use the pass statement to indicate that you need to define attributes and methods for the class later.
8. Write a Python program to iterate through a list of sentences and check if each sentence ends with a question mark. If it does, print the sentence; otherwise, use the pass statement.
9. Implement a Python program that reads a configuration file. For each configuration item, use the pass statement to indicate that further processing is required.

Star programs:

1. Print a simple pattern of increasing asterisks.

```
*
**
***
****
*****
```

2. Print a simple pattern of decreasing asterisks.

```
*****
****
***
**
*
```

3. Print a pattern of right-aligned asterisks with spaces.

```
    *
   **
  ***
 ****
```

```
*****
```

4. Print a pattern of left-aligned asterisks with spaces.

```
******
****
***
**
*
```

5. Print a pyramid pattern with spaces.

```
  *
 ***
*****
```

6. Print an inverted pyramid pattern with spaces.

```
  *
 ***
*****
*******
*********
```

7. Print an upside-down pyramid pattern with spaces.

```
*********
 *******
  *****
   ***
    *
```

8. Print a hollow half pyramid pattern.

```
*
* *
*   *
*     *
* * * * *
```

9. Print a diamond pattern.

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

10. Print a hollow square pattern.

```
* * * * *
*       *
*       *
*       *
* * * * *
```

11. Print a hollow diamond pattern.

```
    *
   * *
  *   *
   * *
    *
```

12. Print an hourglass pattern.

```
*******
 *****
  ***
   *
  ***
 *****
*******
```

13. Print a checkered pattern of stars and spaces.

```
* * * * * *
* * * * * *
* * * * * *
* * * * * *
* * * * * *
```

Number Petterns:

14. Print a ptttern of numbers from 1 to n.

```
1
22
333
4444
55555
```

15. Print a pattern of numbers in a right-angled

```
1
1 2
1 2 3
1 2 3 4
```

16. Print a pattern of numbers in an inverted right-angled triangle:

```
1 2 3 4
1 2 3
1 2
1
```

17. Print a pattern of numbers with alternating rows:

```
1
2 3
4 5 6
7 8 9 10
```

18. Print a pattern of numbers in an inverted pyramid:

```
    1 2 3 4 5
    1 2 3 4
    1 2 3
    1 2
    1
```

19. Print a pattern of numbers in a right-angled triangle (inverted):

```
1
2 1
3 2 1
4 3 2 1
```

20. Print a pattern of numbers in a right-angled triangle with reversed rows:

```
4 4 4 4
3 3 3
2 2
1
```

21. Print a pattern of numbers in an hourglass shape:

```
1 2 3 4 5
2 3 4 5
3 4 5
4 5
5
4 5
3 4 5
2 3 4 5
1 2 3 4 5
```

22. Print a pattern of numbers in a right-angled triangle with descending order:

```
5
54
543
5432
54321
```

23. Print a pattern of numbers in an alternating triangle:

```
1
21
321
4321
54321
```

24. Print a pattern of numbers with a centered diamond shape:

```
1
121
12321
1234321
12321
121
1
```

25. Print a pattern of numbers with a diamond shape:

```
1
123
12345
123
1
```

26. Print a pattern of numbers with a centered triangle:

```
1
212
32123
4321234
```

27. Print a pattern of numbers in a concentric square:

```
11111
12221
12321
```

12221
11111

28. Print a pattern of numbers in a spiral order (with variable dimensions and formatting):
    01 02 03 04 05
    16 17 18 19 06
    15 24 25 20 07
    14 23 22 21 08
    13 12 11 10 09

29. Print a pattern of numbers in a spiral order:
    1 2 3 4
    12 13 14 5
    11 16 15 6
    10 9 8 7

30. Print a pattern of numbers in a spiral order (with variable dimensions):
    1 2 3 4 5
    16 17 18 19 6
    15 24 25 20 7
    14 23 22 21 8
    13 12 11 10 9

31. Print a pattern of numbers in a spiral order (with variable dimensions and formatting):
    01 02 03 04 05
    16 17 18 19 06
    15 24 25 20 07
    14 23 22 21 08
    13 12 11 10 09

Alphabet petterns:

32. Print a simple alphabet pattern:
    A
    B B
    C C C
    D D D D
    E E E E E

33. Print the alphabet pattern in reverse:
    A B C D E
    A B C D

A B C
A B
A

34. Print a simple reverse alphabet pattern:

E E E E E
D D D D
C C C
B B
A

35. Print a simple alphabet pattern:

A B C D E
A B C D
A B C
A B
A

36. Print a simple pattern:

A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y

37. Print a simple pattern:

A B C D E

B C D E F

C D E F G

D E F G H

E F G H I

38. Print a simple pattern:

C

H

K L M N O

R

W

39. Print the following simple pattern:

```
* * * * * *
*           *
*           *
*           *
*  * *  *  *
```

40. Print the following simple pattern:

```
* * * * *
*   f   *
* hgh  *
*   f   *
* * * * *
```

41. Print the following simple pattern:

```
A         F
  B     G
     C
   I   D
J         E
```

42. Print the following simple pattern:

```
A A A A A
B         B
C         C
D         D
E  E  E  E E
```

43. Print the following simple pattern:

```
A b C d E
f  G h  I  j
K l M n O
p Q r  S  t
U v W x Y
```

44. Print the following simple pattern:

```
E D C B A
J  I  H G  F
O N M L K
T  S  R Q P
Y X W VU
```

45. Print the following simple pattern:

```
A     A     A
```

```
    B  B  B
  A B  C  B A
    B  B  B
  A     A     A
```

46. Print the following simple pattern:

```
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
```

47. Print the following simple pattern:

```
        A
      B F
    C G J
  D H K M
E I L N O
```

48. Print the following simple pattern:

```
E
5 5
D D D
4 4 4 4
C C C C
```

Data Structures....

List:

1. Reverse a list in Python ?
2. Remove duplicates from a list in Python ?
3. Find the maximum and minimum elements in a list ?
4. Sort a list in ascending and descending order ?
5. Check if a list is empty in Python ?
6. Find the index of a specific element in a list ?
7. Count the occurrences of an element in a list ?
8. Merge two lists in Python ?
9. Find the sum of all elements in a list ?
10. Remove an element from a list by its value ?
11. Slice a list to get specific elements ?
12. Find the length (number of elements) of a list ?
13. Add an element to the end of a list ?
14. Iterate through a list using a for loop ?
15. Insert an element at a specific position in a list ?
16. Extend a list with elements from another iterable ?
17. Clear all elements from a list ?
18. Create a copy of a list ?
19. Reverse a list in-place (modify the original list) ?
20. Find the index of the first occurrence of an element from a specific position ?
21. Remove and return the last element from a list ?
22. Find the index of the last occurrence of an element in a list ?
23. Check if two lists are equal?

24. Sort a list in-place (modify the original list) ?
25. Find the second largest element in a list
26. Remove elements at even indices from a list:
27. Find the intersection of two lists (common elements):
28. Find the union of two lists (all unique elements from both lists):
29. Remove all occurrences of a specific element from a list:
30. Check if a list is a palindrome (reads the same forwards and backwards):
31. Find the frequency of each element in a list and store it in a dictionary:
32. Find the longest consecutive subsequence in a list:
33. Shuffle a list randomly:
34. Find the majority element in a list (element that appears more than n/2 times, if it exists):
35. Check if a list can be divided into two equal sum subarrays:
36. Generate all possible subsets of a list:
37. Find the missing number in a list containing all numbers from 1 to n except one:
38. Implement a stack using a list:
39. Implement a queue using two stacks.
40. Implement a linked list using a list of dictionaries.

Tuple:

1. Find the index of the first occurrence and last occurrence of a specific element in a tuple
2. Concatenate two tuples:
3. Count the occurrences of an element in a tuple:
4. Check if an element exists in a tuple:
5. Reverse a tuple:
6. Find the maximum and minimum elements in a tuple:
7. Check if all elements in a tuple are the same:
8. Sort a tuple of numbers in ascending order:
9. Find the sum of all elements in a tuple:
10. Check if a tuple contains only unique elements:
11. Multiply a tuple to repeat its elements:
12. Unpack a tuple into individual variables:
13. Convert a tuple of strings into a single string:
14. Merge two tuples into one using the + operator:
15. Create a tuple of tuples (nested tuple):
16. Find the product of all elements in a tuple of numbers:
17. Check if a tuple contains duplicate elements:
18. Remove an element from a tuple and create a new tuple without that element:
19. Find the difference between two tuples:
20. Count the occurrences of each element in a tuple and store the results in a dictionary:
21. Find the common elements between two tuples:
22. Create a tuple from a string:
23. Combine multiple tuples into one:
24. Find the largest element in a tuple of numbers:
25. Find the sum of all elements in a tuple:
26. Create a tuple of even numbers from an existing tuple:
27. Check if a tuple is a subset of another tuple:
28. Check if a tuple is empty without using the len() function:
29. Create a new tuple with elements reversed in order from an existing tuple:
30. Check if a tuple contains only numeric elements:

31. Check if a tuple is a palindrome (reads the same forwards and backwards):
32. Find the tuple with the maximum sum in a list of tuples:
33. Rotate a tuple to the right by k steps:
34. Find the kth smallest and kth largest element in a tuple:
35. Find the contiguous subarray with the smallest sum in a tuple of integers (Minimum Subarray Sum):
36. Find the longest subarray with at most k distinct elements in a tuple of integers:

Set

1. Create an empty set in Python and Add or remove an element to a set.
2. Check if an element exists in a set:
3. Check if one set is a subset of another:
4. Check if two sets are disjoint:
5. Remove all elements from a set:
6. Find the maximum element in a set:
7. Find the minimum element in a set:
8. Convert a list to a set:
9. Remove an element from a set if it exists, otherwise, do nothing:
10. Create a frozen set in Python:
11. Check if a set is empty without using len( ).
12. Remove and return an arbitrary element from a set:
13. Update a set with the elements from another set:
14. Find the sum and average of all elements in a set:
15. Implement set operations (union, intersection, difference):
16. Create a set with unique elements from a list:
17. Check if a set is a superset of multiple other sets:
18. Find the Jaccard similarity between two sets:
19. Find the largest subset of elements in a set that forms an arithmetic progression:
20. Compute the power set of a set:
21. Find the most common element in a set:
22. Find the longest consecutive sequence of elements in a set:
23. Check if two sets have the same elements but in different orders:
24. Find the largest subset of elements in a set that forms an arithmetic progression without using built-in functions:
25. Implement set difference in constant time without using built-in functions.
26. Generate all unique subsets of a set without using built-in functions.
27. Find the longest subarray with distinct elements using a set without using built-in functions.
28. Implement constant-time disjoint set operations without using built-in functions.
29. Find the longest subarray with equal number of 0s and 1s using a set without using built- in functions.
30. Find the longest subarray with at most K distinct elements using a set without using built- in functions.

Dictionary:

1. Create an empty dictionary and add key-value pairs to it:
2. Accessing values associated with a specific key:
3. Adding a key-value pair or updating if the key exists:
4. Removing a specific key-value pair:
5. Dictionary comprehension (filtering):

6. Counting occurrences in a list and storing them in a dictionary:
7. Merging two dictionaries into one:
8. Finding the key with the minimum and maximum value:
9. Sorting a dictionary by its keys in ascending order:
10. Finding the key with the longest value in a dictionary:
11. Determining the number of key-value pairs in a dictionary:
12. Checking if a value exists in a dictionary:
13. Removing a key from a dictionary and getting its value:
14. Reversing key-value pairs in a dictionary:
15. Get a default value if the key doesn't exist:
16. Merge dictionaries using the update method:
17. Sorting a dictionary by values in ascending order/descending order:
18. Checking for the existence of a key and getting its value with a default:
19. Getting a list of keys sorted by their values:
20. Getting a dictionary containing keys and their lengths (useful for strings):
21. Inverting a dictionary (swapping keys and values):

22. Merging dictionaries so that values are added together if keys exist in both dictionaries:
23. Dictionary comprehension with conditions (creating a new dictionary with a conditional statement):
24. Extracting unique values from a dictionary's values:
25. Converting a dictionary to a JSON string:
26. Parsing a JSON string into a dictionary:
27. Implementing a switch-case statement using a dictionary:
28. Finding keys associated with a specific value in a dictionary:
29. Updating multiple values in a dictionary:
30. Getting a dictionary containing keys from one dictionary and values from another:
31. Flattening a nested dictionary:
32. Using a dictionary to implement a simple calculator:
33. How can you safely retrieve a value from a dictionary without raising an error if the key doesn't exist?
34. How can you create a dictionary using a list comprehension?
35. How can you create a deep copy of a nested dictionary?
36. How can you create a dictionary that calculates the frequency of words in a text string?
37. How can you create a dictionary that represents a sparse matrix efficiently?
38. How can you create a dictionary that represents a graph using adjacency lists?
39. How can you create a dictionary that tracks the execution time of functions?
40. How can you create a dictionary that flattens a nested dictionary into a single-level dictionary with compound keys?
41. How can you create a dictionary that groups elements from a list by a function's result?

42. How can you create a dictionary that memoizes the results of a recursive function?
43. How can you create a dictionary that stores and retrieves objects based on unique identifiers?
44. How can you create a dictionary that simulates a cache with a maximum size, removing the least recently used items when the size is exceeded?

List Comprehensions:

1. Create a list of squares from 1 to 10:
2. Filter even numbers from a list:
3. Convert characters in a string to uppercase:

4. Find common elements between two lists:
5. Extract vowels from a string:
6. Find lengths of words in a list of words:
7. Calculate the sum of squares of all even numbers from 1 to 20:
8. Generate a list of the first 10 multiples of 3:
9. Flatten a list of lists into a single list:
10. Find the intersection of two lists of numbers:
11. Filter names that start with the letter "A" from a list of names: 12.Generate a list of all prime numbers less than 50:
12. 13.Filter strings with mor
13. e than five characters from a list of strings: 14.Calculate the factorial of each number in a given list:
14. Create a list comprehension to find the squares of even numbers and cubes of odd numbers in a given list of integers:
15. Generate a list of tuples where each tuple contains an element from two different lists:
16. Calculate the cumulative sum of a list of numbers using a list comprehension:
17. Create a list comprehension to extract the first letter of each word in a sentence:
18. Filter out duplicates from a list using a list comprehension:
19. Create a list comprehension to reverse the order of words in a sentence:

Set Comprehension:

1. Create a set comprehension that generates a set of all even numbers from 1 to 20.
2. Given a list of words, create a set comprehension that generates a set of unique words longer than 3 characters.
3. Generate a set comprehension that extracts the vowels from a given string.
4. Create a set comprehension that calculates the squares of numbers from 1 to 10, but only for numbers that are divisible by 3.
5. Given two sets, A and B, create a set comprehension that generates the union of these sets.
6. Create a set comprehension that generates a set of the first 10 prime numbers.
7. Given two sets, A and B, create a set comprehension that generates the symmetric difference (elements that are in either of the sets, but not in their intersection).
8. Create a set comprehension that extracts all uppercase letters from a given string.
9. Generate a set comprehension that extracts the unique words from a list of sentences.
10. Create a set comprehension that generates a set of the squares of even numbers from 1 to 15.
11. Create a set comprehension that generates a set of all vowels in a given sentence, ignoring case.
12. Generate a set comprehension that extracts unique words longer than 4 characters from a list of sentences.
13. Create a set comprehension that generates a set of the first 10 multiples of 3.
14. Given a list of numbers, create a set comprehension that generates a set of squared values if the number is positive.
15. Generate a set comprehension that extracts unique email domains from a list of email addresses.
16. Question: Create a set comprehension that generates a set of all the distinct odd numbers between 1 and 20.
17. Given a list of sentences, create a set comprehension that generates a set of unique words that start with a vowel.

18. Generate a set comprehension that extracts unique positive even numbers from a given list.
19. Create a set comprehension that generates a set of unique lowercase characters from a given string.
20. Given a list of tuples representing points in a 2D plane, create a set comprehension that generates a set of unique x-coordinates.

Dictionary comprehension

1. Create a dictionary comprehension that squares each number in a given list and uses the number as the key.
2. Given two lists, one containing names and the other containing ages, create a dictionary comprehension to map names to ages.
3. Generate a dictionary comprehension that filters out names longer than 5 characters from a list of names.
4. Create a dictionary comprehension to count the frequency of each character in a given string.
5. Given a list of tuples representing words and their lengths, create a dictionary comprehension to construct a dictionary where words are keys and lengths are values.
6. Create a dictionary comprehension that extracts all words starting with a vowel from a given list of words.
7. Given a list of sentences, create a dictionary comprehension that counts the number of words in each sentence.
8. Generate a dictionary comprehension that calculates the square of numbers from 1 to 10, but only for numbers that are divisible by 3.
9. Given a list of strings, create a dictionary comprehension that maps each string to its length and adds a key-value pair only if the length is even.
10. Create a dictionary comprehension to convert a list of integers into a dictionary where keys are the integers themselves, and values are their squares.
11. Question: Given two lists, one containing student names and the other containing their corresponding scores, create a dictionary comprehension to map names to scores. Only include students with scores greater than or equal to 80.
12. Generate a dictionary comprehension that extracts unique first letters from a list of words and counts their occurrences.
13. Create a dictionary comprehension that extracts all words longer than 4 characters from a given list of sentences and uses the words as keys with their lengths as values.
14. Given a list of email addresses, create a dictionary comprehension that maps the username (before '@') to the domain.
15. Create a dictionary comprehension that extracts all uppercase letters from a given string and counts their occurrences.
16. Question: Given a list of numbers, create a dictionary comprehension to group the numbers by whether they are even or odd.
17. Generate a dictionary comprehension that combines two lists into a dictionary where elements from one list are keys and elements from the other list are values.
18. Create a dictionary comprehension to extract all words from a list of sentences and count their occurrences in the entire text.
19. Given a list of strings, create a dictionary comprehension that maps each string to its length and adds a key-value pair only if the length is odd.
20. Generate a dictionary comprehension to calculate the factorial of numbers from 1 to 5.

Shuffling

1. Given a list of numbers, shuffle its elements.
2. Create a program that simulates picking a random card from a deck.
3. Given a sentence, shuffle its words.
4. Given a list of numbers, shuffle the even and odd numbers separately and then merge them.
5. Given two lists, shuffle them together such that one element from the first list is followed by one from the second, and so on.
6. Given a list of numbers, shuffle them such that no two even numbers or two odd numbers come together.
7. Given a list of names, create random pairs.
8. Given a string, shuffle its characters but ensure vowels remain in the same position.
9. Shuffle a deck of cards and deal n hands of m cards each.
10. Given a list of numbers, shuffle it such that no number is next to its original neighbors.
11. Given a list, shuffle its elements such that elements which are the same are at least 2 positions away from each other.
12. Given two decks of cards, shuffle them together, ensuring that no two cards from the same deck come together.
13. Given a 2D matrix, shuffle its rows but maintain the relative order of elements within each row.
14. Given a paragraph (multiple sentences), shuffle the sentences.

SWAPPING

1. Given two numbers, swap their values without using a third variable.
2. Given two strings, swap their values.
   Example Input: str1 = "hello", str2 = "world"
   Output: ("world", "hello")
3. Given a list, swap its first and last elements.
   Example: Input: lst = [1, 2, 3, 4, 5]
   Output: [5, 2, 3, 4, 1]
4. Given a string, swap the characters at positions i and j
   Example: Input: s = "hello", i = 1, j = 3
   Output: "hlleo"
5. Given a list, swap elements at even indices with elements at odd indices.
6. Given a list, swap every pair of elements and then reverse the entire list.
   Example: Input: lst = [1, 2, 3, 4, 5, 6]
   Output: [6, 5, 4, 3, 2, 1
7. Given two lists of equal length, swap the i-th element of the first list with the i-th element of the second list.
8. Given two lists, swap their middle elements. If a list has an even number of elements, consider the latter middle.
9. Given a list, repeatedly swap the largest number in the list with its right neighbor until it reaches the end of the list.
10. Given a list, swap every element with its mirrored position. For instance, the first element is swapped with the last, the second with the second last, and so on.
11. Given two lists of the same length, swap their elements in an alternating pattern: one from the first list, one from the second, and so on.
12. Check if it's possible to make a string a palindrome by swapping exactly two characters.

SORTING

1. Given a list of numbers, sort them in ascending order.
2. Given a list of strings, sort them alphabetically.
3. Given a list of strings, sort them by their length.
4. Given a list of numbers, sort them in descending order.
5. Given a list of tuples where each tuple contains two elements, sort the list based on the second element of each tuple.
6. Given a list of words, sort them based on their last character.
7. Given a list of numbers, sort them based on their frequency in the list. If two numbers have the same frequency, sort them in ascending order.
8. Given a matrix (2D list), sort its rows based on the sum of elements in each row.
9. Given a list of numbers and a reference number, sort the list based on the absolute difference between each number and the reference number.
10. Given a list of integers and strings, sort them such that all strings are on the left and all integers are on the right.
11. Given a list of words, sort them based on the count of vowels in each word. If two words have the same vowel count, maintain their relative order.
12. Given a list of numbers, sort the list such that all even numbers are on the left and all odd numbers are on the right, with both sections sorted in ascending order
13. Given a matrix (2D list), sort its columns based on the sum of elements in each column.
14. Given a list of numbers, sort them such that all prime numbers are on the left and non-prime numbers on the right, with both sections sorted in ascending order

Strings:

1. Reverse a string
2. Count vowels
3. Capitalize First Letter of Words
4. Check palindrome
5. Count Occurrences of Substring
6. Length of string
7. Convert to Uppercase
8. Convert to Lowercase
9. Concatenate Strings
10. Check if Character Exists
11. Print First and Last Character
12. Add an Exclamation to a String
13. String Repetition
14. String Spaces
15. Replace a Character
16. Display Your Name
17. Simple Greeting
18. Count the number of times the letter 'e' appears in a given string.
19. Simple String Concatenation
20. Is the String Empty?
21. Write a program to remove all vowels from a given string.
22. Count Words
23. Check if String is a Palindrome (ignoring spaces)
24. Given a string, display it in a pyramid format.

25. Encode String Write a program that encodes a string by replacing each letter with the next letter (i.e., replace 'a' with 'b', 'b' with 'c', ..., 'z' with 'a').
26. Find the Longest Word
27. Determine if two given strings are anagrams (i.e., they contain the same characters in any order).
28. Count the number of consonants in a given string.
29. Extract all the numbers from a given string and display them.
30. Find All Occurrences of Substring
31. Implement a basic string compression using counts of repeated characters. For example, the string "aabcccccaaa" would become "a2b1c5a3". If the compressed string is not smaller than the original string, return the original string.
32. Rotate a string to the right by a given number of positions
33. Count the occurrence of each character in a string.
34. Given two strings, interleave their characters starting with the first string. If one string is longer than the other, append the remaining characters of the longer string at the end
35. Check if a string is a subsequence of another string. A string s1 is a subsequence of s2 if the characters of s1 appear in the same order in s2, though not necessarily consecutively.
36. Check if a given string can be constructed by taking a substring and repeating it
37. First Non-Repeating Character Given a string, find the first non-repeating character in it.
38. Determine if the input string has valid parentheses. For example, "()" and "()[]{}" are valid, but "(]" and "([)]" are not.
39. Given a string, reverse the order of its words
40. Find the first character in a string that repeats
41. Given a string s consisting of some words separated by spaces, return the length of the last word in the string. If the last word does not exist,
42. return 0. A word is defined as a sequence of non-space characters.
43. Remove all adjacent duplicate characters from a string. Continue removing until no more adjacent duplicates are found.
49. Given a list of strings, find the longest common prefix among them.

```
Example :Input: ["flower", "flow", "flight"]
# Output: "fl"
```

50. Implement a function that converts a string to an integer. The function should handle positive/negative sign and ignore leading/trailing whitespaces.
51. Do not use the built-in int() function.
52. Given a Roman numeral, convert it to its integer value.
53. Given an integer n, generate the nth term of the "count and say" sequence.
54. Check if a given string can become a palindrome by deleting at most one character from it.
55. Given an array of strings, group anagrams together
56. Given two strings, check if they are one edit distance apart, i.e., they can become equal by adding, deleting, or replacing exactly one character.
57. Given two non-negative integers num1 and num2 represented as strings, return the product of num1 and num2, also represented as a string.
58. Given a non-empty string containing only digits, determine the total number of ways to decode it into alphabets (e.g., '1' can be decoded to 'A', '26' can be decoded to 'Z').

59. Compress a string such that 'AA' becomes 'A2' but single letters should remain unchanged. Return the shortest compressed string possible.
60. Display a String
61. Access a Character in a String

String methods:

1. capitalize()
   Converts the first character of a string to uppercase.
2. casefold()
   Converts string to lowercase, it's more aggressive than lower() and can modify strings to be more suitable for caseless comparisons.
3. center()
   Returns a centered string.
4. count()
   Counts the occurrences of a substring in string.
5. encode()
   Returns an encoded version of the string.
6. ndswith()
   Checks if the string ends with the specified suffix.
7. expandtabs()
   Replaces tab characters with spaces.
8. find()
   Finds a substring in a string. Returns -1 if not found.
9. format()
   Formats the string.
10. index()
    Like find(), but raises an exception if not found.
11. isalnum()
    Checks if the string has only alphanumeric characters.
12. isalpha()
    Checks if the string has only alphabetic characters.
13. isdecimal()
    Checks if the string has only decimal characters.
14. isdigit()
    Checks if the string has only digits.
15. isidentifier()
    Checks if the string is a valid identifier.
16. islower()
    Checks if all the letters in the text are in lowercase.
17. isnumeric()
    Checks if the string has only numeric characters.
18. isprintable()
    Checks if the string is printable.
19. isspace()
    Checks if the string has only whitespace characters.
20. istitle()
    Checks if the string is in title case
21. isupper()
    Checks if all the letters in the text are in uppercase.
22. join()
    Concatenates a list of strings.

23. ljust()
    Returns a left-justified version of the string.

24. lower()
    Converts the string to lowercase.
25. lstrip()
    Removes leading whitespace.
26. partition()
    Divides a string based on a separator.
27. replace()
    Replaces a substring with another.
28. rfind()
    Finds a substring in a string, starting from the right. Returns -1 if not found.
29. rindex()
    Like rfind(), but raises an exception if not found.
30. rjust()
    Returns a right-justified version of the string.
31. rpartition()
    Like partition(), but starts from the right.
32. rsplit()
    Splits a string into a list, starting from the right.
33. rstrip()
    Removes trailing whitespace.
34. split()
    Splits a string into a list.
35. splitlines()
    Splits a string by line breaks.
36. startswith()
    Checks if the string starts with a specific substring.
37. strip()
    Removes both leading and trailing whitespace.
38. swapcase()
    Swaps cases, lowercase becomes uppercase and vice versa.
39. title()
    Converts the first character of each word to uppercase.
40. upper()
    Converts a string into uppercase.
41. zfill()
    the string with a specified number of 0 values at the beginning.
42. removeprefix() and removesuffix()
    Removes a specified prefix or suffix from a string.
43. maketrans() and translate()
    These two methods are used together for character mapping. The maketrans() method returns a translation table. The translate() method returns a string where some specified characters are replaced with the character described in a dictionary, or in a mapping table.
44. isascii()
    Checks if all the characters in the text are ascii characters.
45. isprintable()
    Checks if all the characters in the text are printable:

46. rstrip()
    Removes trailing characters (based on the string argument passed).

47. lstrip()
    Removes leading characters.
48. casefold()
    Returns a string where all the characters are lower case.
49. format_map()
    Formats specified values in a string:

50. join()
    Takes all items in an iterable and joins them into one string.
51. ljust()
    Returns a left justified version of the string.
52. rjust()
    Returns a right justified version of the string.
53. strip()
    Removes any leading (spaces at the beginning) and trailing (spaces at the end) characters.
54. swapcase()
    Swaps cases, lowercase becomes uppercase and vice versa.

Slicing:

1. Given a string, slice it to extract the first 3 characters.

   Input:"Vcube"
   Output:" Vcu"

2. Reversing a List using slicing
   Input: [1,2,3,4]
   Output: [4,3,2,1]

3. Slicing a Tuple

   Input:(10,11,12,13,14)
   Output:(10,11,12)

4. Modify a List Using Slicing

   Input: [100,200,300,400]
   Output: [100,500,300,400]

5. Extract Even-Indexed Characters from a String

   Input:"Vcube Solutions"
   Output:"VueSltns"

6. Slicing a List of Lists

   Input: [1 ,[2,3,4],5,6]
   Output: [3,4]

7. Slicing a String in Reverse

   Input:”Vcube”
   Output:”ebucV”

8. Slicing with a Step of 3

   input:”Python”
   Output:”Ph”

9. Reversing a List of Strings.

   Input: ["vcube”,”python”,”java”,]
   Output["avaj”,”nohtyp”,’ebucv”]

10. Slicing a String with a Step Value

    Input:”Vcube Solution”
    Output:”VbSun”

11. Concatenating Lists by using slicing.
12. Remove Duplicates from a list using slicing.

    Input:[1,2,3,1,4,5,2]
    Output: [1,2,3,4,5]

13. Write a pyhton program to reverse the order of words in a given string by slicing and joining.

    Input:”Python”
    Output:”nohtyP”

14. Flattening Nested Lists by using slicing.
15. Finding the Maximum Value in a given list by slicing.

    Input: [10,11,12,13,14,15]
    Output:15

16. Write a python program that count the number of occurrences of substring in string by slicing .

    Input:”Raja” substring:”aja”
    Output:1

17. Given a string `s`, determine if it is a palindrome by slicing and checking.
18. Given two dictionaries `dict1` and `dict2`, merge them together by slicing and combining selected keys

    Example

    Input: dict1 = {'a': 1, 'b': 2, 'c': 3}, dict2 = {'x': 10, 'y': 20, 'z': 30}, keys1 = ['a', 'b'], keys2 = ['y', 'z']
    # Output: {'a': 1, 'b': 2, 'y': 20, 'z': 30}

Functions

1. Can you provide a simple function in Python without arguments and a return statement?
2. How would you define a function with parameters in Python? Can you give an example?
3. How can you create a function with a default parameter in Python? Can you provide an example?
4. How would you write a function with a docstring in Python? Can you show an example?

For Scope:

5. Can you provide an example to demonstrate local scope in Python?
6. How would you demonstrate enclosing scope in Python with an example?
7. Can you give an example to illustrate global scope in Python?
8. How can you show built-in scope with an example in Python?

Modifying and Accessing Variables:

9. How would you modify a global variable from within a function in Python? Can you provide an example?
10. Can you show how to access a global variable from within a function in Python?
11. How can you modify a nonlocal variable in a nested function in Python? Can you give an example?
12. How would you access a nonlocal variable in a nested function in Python? Can you provide an example?

Anonymous Functions (Lambda Functions):

1. Can you provide a lambda function to square a number in Python?
2. How can you create a lambda function to add two numbers in Python? Can you give an example?
3. How would you write a lambda function to find the maximum of two numbers in Python? Can you show an example?
4. How can you create a lambda function to check if a number is even in Python? Can you provide an example?
5. Can you demonstrate a lambda function to extract the last character from a string in Python?
6. How can you use a lambda function to calculate the square root of a number using the `math` module in Python? Can you show an example?
7. How would you write a lambda function to sort a list of tuples based on the second element in Python? Can you provide an example?

8. How can you create a lambda function to filter even numbers from a list in Python? Can you give an example?
9. Can you provide a lambda function with a conditional expression to categorize ages in Python?
10. How can you write a lambda function to calculate the factorial of a number using recursion in Python? Can you demonstrate this with an example?

Use cases for lambda functions

1. How can you sort a list of tuples by the second element in Python? Can you provide an example?
2. How would you filter even numbers from a list in Python? Can you give an example?
3. How can you calculate the square of numbers in a list in Python? Can you demonstrate this with an example?
4. How would you create a simple calculator in Python? Can you provide an example?
5. How can you use lambda with the `sorted` function for custom sorting in Python? Can you show an example?
6. How do you calculate factorial using recursion in Python? Can you provide an example?
7. How can you combine lists element-wise in Python? Can you demonstrate this with an example?
8. How would you remove duplicates from a list in Python? Can you give an example?
9. How can lambda functions be used in GUI applications, such as Tkinter, in Python? Can you provide an example?
10. How do you perform custom sorting of objects in a list in Python? Can you show an example?

Recursion

1. Write a Python function to calculate the factorial of a given number using recursion.

2. Write a Python function to generate the nth Fibonacci number using recursion.

3. Write a Python function to calculate the sum of digits of a given number.

4.Print the Pattern by using recursive Function
n=5
Output:54321012345

5.Find the sum of the elements in the given list by using recursive Function
L= [1,2,3,4,5,6]
Output: 21

6.Convert the nested list into single list by using recursive function
L= [1, [2,3],4,[5], [6,7,8,9],10]
Output: [1,2,3,4,5,6,7,8,9,10]

7.Check the given number is Prime or not by using recursive function
Input:5
Output: Prime number

8.Print n even numbers by using recursive Function
Input:5
Output:0,2,4,6,8.

9.Print n odd numbers by using recursive Function
Input:5
Output:1,3,5,7,9

10.Find the Sum of the given number by using recursive Function

Input:12345
Output:15

Stack overflow and tail recursion optimization

1. How would you implement basic tail recursion in Python?
2. Can you provide an example of a stack overflow occurring without optimization in Python?
3. How do you optimize tail recursion with an accumulator in Python? Can you show an example?
4. Can you demonstrate the calculation of the Fibonacci sequence without optimization in Python?
5. How can a stack overflow occur in a deep recursive function in Python? Can you provide an example?
6. How would you implement tail recursion in the calculation of a factorial in Python?

Function Arguments

Default arguments

1. How do you define a function with a default argument in Python?
2. How can you define a function with multiple default arguments in Python?
3. How do you handle default arguments with lists in Python?
4. How do you use `None` as a default argument in Python to avoid issues with mutable default arguments?

Keyword arguments

1. Can you provide an example of basic keyword argument usage in Python?
2. How do you use keyword arguments with default values in Python? Can you give an example?
3. Can you demonstrate keyword arguments with input validation in Python?
4. How would you use keyword arguments for file I/O operations in Python?
5. Can you show an example of a function that uses keyword arguments with multiple parameters in Python?

Concatenate Strings:

```
# Example usage
words = ["Hello", ", ", "World", "!"]
sentence =
concatenate_strings(*words)
print(sentence)
```

Find Maximum Number:

```
# Example usage
numbers = [12, 45, 67, 23, 56]
max_value = find_max(*numbers)
print(f"The maximum number in {numbers} is {max_value}")
```

Calculate Average:

```
# Example usage
grades = [85, 92, 78, 90, 88]
average_grade = calculate_average(*grades)
print(f"The average grade is:
{average_grade:.2f}")
Print Elements with a
Separator: # Example usage
words = ["apple", "banana", "cherry",
"date"] separator = ", "
print_with_separator(separator, *words)
```

Variable-length keyword argument dictionaries (**kwargs)

1. How do you use **kwargs in Python, and can you provide a simple example?
2. Can you demonstrate how to calculate an average using **kwargs in Python?
3. How would you create a configurable function using **kwargs in Python?
4. Can you provide an example of an HTML element generator using **kwargs in Python?

Higher-Order Functions:(Functional Programming and Higher-Order Functions)

- Functions as first-class objects

1. Can you provide an example of using functions as arguments in Python?
2. How can functions be used as return values in Python? Can you provide an example?
3. How can functions be stored in data structures in Python? Can you give an example?
4. How are functions used as decorators in Python? Can you demonstrate this with an example?

Passing functions as arguments

1. Can you provide an example of using a basic function as an argument in Python?
2. How would you use functions for data transformation in Python? Can you give an example?
3. Can you demonstrate using a function for sorting in Python?

Returning functions from functions

1. How can you return a function in Python to perform arithmetic operations?
2. Can you provide an example of returning a function in Python to filter a list?
3. How would you use a function to generate a sequence in Python and return it?

Built-in higher-order functions (e.g., `map`, `filter`, `reduce`)

1. How can you square all elements in a list in Python?
2. Can you demonstrate how to convert a list of strings to uppercase in Python?

Filter Function:

1. How would you filter even numbers from a list in Python? Can you provide an example with input and output?
2. Can you demonstrate how to filter names with more than 5 characters from a list in Python? Please provide an example with input and output.

General Higher-Order Functions:

How can you create a custom higher-order function in Python to apply a function to elements greater than a threshold in a list? Can you provide an example with input and output?

Decorators

1. How do you create a basic decorator in Python?
2. How can you create a decorator with arguments in Python?
3. How would you create a timing decorator in Python?
4. How can you create a logging decorator in Python?
5. How do you implement a memoization decorator (caching) in Python?

Applying decorators to functions

1. Logging Decorator: This decorator logs the input arguments and the return value of a function.
2. Timing Decorator: This decorator measures the time taken by a function to execute.
3. Authorization Decorator: This decorator checks if a user is authorized to access a specific function.

Common use cases for decorators (e.g., logging, authentication)

1. How would you create an authentication decorator in Python?
2. Can you demonstrate how to create a caching decorator in Python?
3. How do you implement an error handling decorator in Python?

Closures

1. How can you define a simple nested function in Python?
2. Can you demonstrate how to return a nested function in Python?
3. How do you use parameters in a nested function in Python?

Accessing variables in outer function scopes

1. How would you access an outer variable in a nested function in Python?
2. Can you provide an example of modifying an outer variable in a nested function in Python?

Closure properties

1. How do you create a closure in Python?
2. Can you demonstrate a closure with mutable state in Python?

Call stack

1. How does a recursive function use the call stack in Python?
2. What happens when a recursive function causes a call stack overflow in Python?

Generator Functions

1. How do you create a simple generator in Python using the `yield` statement?

Infinite Sequence Generator:

2. Can you explain how Python's standard library functions are used for File I/O with `open()`?
3. How do you perform mathematical operations using the `math` module in Python?
4. Can you demonstrate how to work with Date and Time using the `datetime` module in Python?
5. How would you

6. work with JSON data using the `json` module in Python?
7. How can you use the `re` module in Python for working with Regular Expressions?
8. What are some commonly used built-in functions in Python, such as `len()`, `range()`, `zip()`, `sum()`, and `input()`?
9. How does the `range()` function generate a range of numbers in Python?
10. Can you demonstrate how to combine lists using the `zip()` function in Python?
11. How do you use the `sum()` function to calculate the sum of elements in a list in Python?
12. How can you use the `input()` function to get user input in Python?

Custom Function Libraries:

1. How do you create and use custom function libraries or modules in Python?
2. Can you provide examples of custom function libraries for Math Operations, Data Processing, and File Handling?
3. Can you explain the process of packaging and distributing functions via PyPI (Python Package Index) in Python?

Class and Object:

1. Create a class:
2. create a object:
3. simple calculator using class:
4. Bank Account using class :
5. student class
6. car class:
7. Library Book Class:
8. Create a class Rectangle that calculates the area and perimeter of a rectangle.
9. Create a class Person that stores and displays information about a person's name and age.
10. Create a class Calculator that performs basic arithmetic operations:addition, subtraction, multiplication, and division.
11. Create a class Circle that calculates the area and circumference of a circle.
12. Create a class Student that represents a student's information, including name,roll number, and marks in three subjects. Calculate the average marks and display the student's details.
13. Create a class BankAccount that represents a simple bank account. Implement methods for deposit, withdrawal, and checking the account balance.
14. Create a class Car that simulates a car with attributes like make, model, and current speed. Implement methods to accelerate and brake the car.
15. Create a class Book that represents a book with attributes like title, author, and publication year. Implement a method to display book details.
16. Create a class Employee to represent employee information, including name, employee ID, and salary. Implement a method to give a salary raise and display employee details.
17. Create a class Bank that represents a bank with attributes like name and location. Implement methods to add and display account holders' names.

18. Create a class TemperatureConverter that converts temperatures between Celsius and Fahrenheit. Implement methods for conversion in both directions.
19. Create a class Vector that represents a 2D vector with attributes x and y. Implement methods to calculate the magnitude and display the vector.
20. Create a class Shop that represents an online shop. Implement methods for adding products to the shop and displaying the list of available products.

Inheritance

1. Create a base class Animal that has a method sound. Extend this class with subclasses Dog and Cat where both provide their specific implementations for the sound they make
2. Develop a base class Vehicle with a method type_of_vehicle. Extend this class with subclasses Bike and Car, and provide specific types for each vehicle.
3. Design a base class Fruit with an attribute taste. Extend this class with subclasses Apple and Orange, setting their specific taste in their constructors.
4. Create a base class Bird that has a method fly. Extend this class with subclasses Sparrow and Ostrich. While the Sparrow can fly, the Ostrich cannot.
5. Develop a base class Shape with a method area. Extend this class with subclasses Square and Circle. The subclasses should calculate their specific areas based on their attributes.
6. Design a base class Person with attributes name and age. Extend this class with subclasses Student and Teacher. The Student class should have an additional attribute grade, and the Teacher class should have an attribute subject.
7. Design a class hierarchy for a university system. Begin with a base class UniversityMember with attributes name and id_number. Extend this class with subclasses Student and Professor. The Student class should have additional attributes like major and year, while the Professor class should have department and publications.
8. Create a banking system. Start with a base class Account with methods deposit, withdraw, and balance. Extend this class with subclasses SavingsAccount and CheckingAccount.
   SavingsAccount should have an added interest rate, while CheckingAccount may have an overdraft limit.
9. Develop a system to manage products in a store. Design a base class Product with attributes name and price. Extend this class with DiscountedProduct, which should apply a discount percentage to the product's price.
10. Design a class hierarchy for a transportation system. Create a base class Vehicle with attributes make and model, and methods start and stop. Extend this class with subclasses Car and Boat. The Car class should have an additional method honk, while the Boat class should have a method anchor.
11. Develop a system to manage different types of employees in a company. Start with a base class Employee with attributes name and salary, and a method display_salary. Extend this with subclasses Manager and Intern. A Manager has an additional bonus to their salary, while an Intern has a stipend instead of a regular salary.
12. Design a library system with a base class Book that has attributes title and author. Extend this class with subclasses Fiction and NonFiction. The Fiction class should have an additional attribute genre, and the NonFiction class should have an attribute subject.

13. Design an e-commerce system with product ratings and reviews. Start with a class Product with attributes name and price. Extend this class with RatedProduct, which has attributes to keep track of average rating and reviews.
14. Design a class hierarchy for a zoo. Create a base class Animal with attributes name, age, and methods sound and eat. Extend this class with two subclasses Mammal and Bird. Further extend Mammal with Lion and Dolphin and extend Bird with Eagle and Penguin. Each subclass should provide its specific implementation for the sound it makes.
15. Design a digital library system. Start with a base class Content having attributes title and size. Extend this class with subclasses Book and Video. The Book class should have additional attributes like author and number_of_pages, while the Video class should have duration and director.

Abstraction

1. Create a class Smartphone that abstracts the idea of a smartphone. It should have methods to call, text, and browse but will not have the inner workings of these methods defined. Extend this class with a specific smartphone brand (e.g., iPhone) that implements the methods.
2. Design an abstract class Vehicle that represents general vehicles. It should have methods to start_engine, stop_engine, and drive. Then, implement a class Car that inherits from Vehicle and provides a concrete implementation for these methods.
3. Design an abstract class Appliance that represents general household appliances. This class should have methods turn_on and turn_off. Extend this class with two specific appliances: Refrigerator and Fan, implementing the methods.
4. Design an abstract class Instrument that represents musical instruments. It should have a method play. Implement two specific instruments: Guitar and Drum, providing specific sounds when played.
5. Create an abstract class Document that represents written documents. It should have methods read and write. Implement two specific types of documents: Letter and Report, and give specific implementations for reading and writing.

6. Design an abstract class Payment that outlines the structure for making payments. It should have methods authorize, pay, and refund. Now, implement two specific payment methods: CreditCard and PayPal, each having its specific way of handling the payment operations.
7. Develop an abstract class Shape that represents geometric shapes. It should have methods area and perimeter. Implement two specific shapes: Rectangle and Circle, providing calculations for their area and perimeter.
8. Design an abstract class FileHandler that outlines the blueprint for handling files. It should have methods open, read, write, and close. Implement specific handlers for different types of files: TextFile and BinaryFile.
9. Develop an abstract class NotificationService representing a service to send notifications. It should have methods authenticate, send and logout. Implement two specific services: EmailService and SMSService, which use different protocols for sending notifications.
10. Design an abstract class ECommercePlatform which defines a blueprint for an e-commerce system. The class should have abstract methods for list_product, add_to_cart, checkout, and make_payment. Implement two subclasses Shopify and Magento, providing specific implementations for these methods.
11. Develop an abstract class SocialMediaPlatform that outlines functionalities for a social media service. The class should have abstract methods for post_content,

comment, like, and share. Implement subclasses for two hypothetical platforms: Facegram and Twitbook, each with distinct ways of handling these operations.

12. Design an abstract class GamingPlatform that simulates the workings of a gaming console. It should have methods for load_game, play, save_game, and shutdown. Now, simulate two platforms: PlayBox and GameStation, each having unique ways of executing the operations.
13. Develop an abstract class StreamingService representing online media streaming. The class should have methods for login, search_content, stream, and logout. Now, create two specific streaming platforms: Netflex and PrimeWatch to simulate their unique behaviors.
14. Create an abstract class MusicPlayer that simulates basic operations for playing audio. The class should have methods for play_song, pause, next_track, and previous_track. Develop two specific audio players: SpotifyPlayer and AppleMusicPlayer that have distinct behaviors for these operations.
15. Construct an abstract class VehicleControlSystem simulating a vehicle's control system. The class should have methods for start_engine, stop_engine, turn_on_headlights, and play_radio. Create two subclasses CarControlSystem and MotorbikeControlSystem to represent their specific control behaviors.

Polymorphism

1. Define two classes, Rectangle and Circle. Both classes should have a method area that calculates and returns their respective areas. Create objects of these classes and add them to a list. Iterate over this list to print the area for each shape.
2. Create classes Car and Bike. Both should have a method num_of_wheels that returns the number of wheels they have. Instantiate objects from these classes, add them to a list, and then iterate over this list to print the number of wheels each vehicle has.

3. Design a system with a base class Vehicle that has a method capacity. Extend this class into two subclasses: Bus and Car. While Bus can carry multiple passengers, Car is meant for fewer passengers. Instantiate objects of these classes, add them to a list, and loop over this list to display the capacity of each vehicle.
4. Construct a class hierarchy for animals with a base class Animal. This class should have a method diet which returns the general diet of the animal. Extend this with two subclasses: Carnivore and Herbivore. Further, extend Carnivore with a class Tiger and Herbivore with a class Cow. Instantiate objects and display their diets.
5. Design a base class Shape with a method perimeter that returns the perimeter of the shape. Extend this class with subclasses: Triangle and Rectangle. The perimeter calculation varies for each shape. Instantiate objects from these subclasses, place them in a list, and then loop over this list to display the perimeter of each shape.
6. Design a system where you have a base class Instrument that has a method sound. Extend this class with three subclasses: Guitar, Piano, and Drums. Each subclass should provide its own implementation for the sound method. Instantiate objects of these subclasses, add them to a list, and loop over this list to play the sound of each instrument.
7. Create a class hierarchy for a library system. Start with a base class Book that has a method genre. Extend this with two subclasses: Fiction and NonFiction. Further extend Fiction with classes SciFi and Romance, and NonFiction with History and Biography. Instantiate objects and determine their genre.
8. Design a class hierarchy for electronic devices. Create a base class Device with a method purpose. Extend this with ComputingDevice and CommunicationDevice. Further extend ComputingDevice with Laptop and Desktop, and

CommunicationDevice with MobilePhone and Landline. Instantiate these and display their purposes.

9. Design a simple simulation of a zoo with different animal types and behaviors. Start with a base class Animal that has methods sound, eat, and move. Extend this with classes Bird, Mammal, and Fish. Each of these subclasses should override the methods in unique ways.
   Further, extend Bird with Eagle and Penguin, Mammal with Lion and Kangaroo, and Fish with Shark and Clownfish. Instantiate these subclasses and demonstrate their behaviors.
10. Design a system to manage different types of documents. Start with a base class Document that has methods content_type and print_content. Extend this with TextDocument, PDFDocument, and ImageDocument. Each subclass should handle its unique content type and print method. Instantiate these classes and demonstrate their content handling and printing capabilities.
11. Create a simulation of geometric transformations on shapes. Start with a base class Shape with methods translate and scale. Extend this with Circle and Rectangle. Both subclasses should have their own methods to modify the position or dimensions, influenced by the base class's transformations.
12. Design a system to simulate different types of printers in an office setting. Start with a base class Printer with methods connect and print_document. Extend this class with subclasses: LaserPrinter, InkjetPrinter, and DotMatrixPrinter. Each subclass should demonstrate unique behaviors while connecting and printing. Instantiate these classes and demonstrate the printer operations.
13. Design a simulation where different storage devices can read and write data. Begin with a base class StorageDevice having methods read and write. Extend this with classes: HardDrive, SSD, and USBStick. Each subclass should provide specific reading and writing speeds. Instantiate and demonstrate data operations on these storage devices.
14. Design a class hierarchy for different payment methods in an e-commerce system. Begin with a base class Payment with methods authorize and charge. Extend this with CreditCard, DebitCard, and PayPal. Each subclass should implement authorization and charging uniquely. Instantiate these classes and demonstrate a payment flow.
15. Create a simulation of a multimedia system. Begin with a base class Media with methods play and stop. Extend this with subclasses AudioFile, VideoFile, and ImageFile. Each subclass should simulate the playing and stopping of its content type in unique ways.
    Instantiate these classes and demonstrate their behaviors.
16. Design a logging system. Start with a base class Logger having methods info, error, and warning. Extend this with ConsoleLogger, FileLogger, and NetworkLogger. Each subclass should have its own way of logging messages to its medium. Instantiate and demonstrate logging actions using these classes.

Method-Overriding

17. Design a base class Shape with a method area that returns 0. Create two subclasses, Circle and Square, and override the area method to return the appropriate area for each shape.

18. Design a base class Animal with a method sound that returns "Animal makes a sound". Develop two subclasses, Dog and Cat, and override the sound method to produce the sound each animal generates.
19. Construct a base class Vehicle with a method description that returns "This is a vehicle". Generate two subclasses, Car and Bike, and override the description method to return more specific descriptions for each type of vehicle.
20. Design a base class Bird with a method fly that returns "A bird can fly". Create two subclasses, Ostrich and Penguin, and override the fly method for these birds since they cannot fly.
21. Design a base class Device with a method power_on that returns "Device is powering on". Construct two subclasses, Laptop and Smartphone, and override the power_on method to give a customized power-on message for each device.
22. Create a class BankAccount with methods deposit, withdraw, and get_balance. The get_balance method should simply return the balance. Now, derive a subclass InterestAccount that overrides the get_balance method to include interest on the balance.
23. Build a class OnlineGame with methods start_game and end_game. The start_game method should return "Starting online game..." and end_game should return "Ending online game...". Derive a subclass BattleRoyaleGame that overrides the start_game method to specify that players are joining a battle royale match.
24. Establish a class Vehicle with a method fuel_up that returns "Fueling up vehicle...". Create a subclass ElectricCar that overrides the fuel_up method to indicate that the vehicle is charging its battery.
25. Develop a class Document with a method save that saves the document. Now, derive a subclass EncryptedDocument that overrides the save method. The overridden method should first encrypt the content before saving.
26. Design a class ECommerceSite with methods add_to_cart, remove_from_cart, and checkout. Now, derive a subclass PremiumECommerceSite which overrides the checkout method. In the overridden method, apply a discount for premium users before proceeding to checkout.
27. Establish a class Vehicle with a method fuel_up that returns "Fueling up vehicle...". Create a subclass ElectricCar that overrides the fuel_up method to indicate that the vehicle is charging its battery.
28. Develop a class Document with a method save that saves the document. Now, derive a subclass EncryptedDocument that overrides the save method. The overridden method should first encrypt the content before saving.

29. Design a class ECommerceSite with methods add_to_cart, remove_from_cart, and checkout. Now, derive a subclass PremiumECommerceSite which overrides the checkout method. In the overridden method, apply a discount for premium users before proceeding to checkout.
30. Design a class StringManipulator that holds a string. Overload the / operator such that it splits the string by another string passed to it.
31. Create a class Polygon that represents a polygon through a list of (x, y) coordinates. Overload the & operator to find the intersection of two polygons.
32. Design a class SafeList that extends the behavior of a regular list but overloads the [] operator (using _getitem and _setitem_). When trying to access an index out of range, it should print an error message instead of throwing an exception.

33. Design a class SparseVector to represent vectors where most of the elements are zero. Overload the * operator to compute the dot product between two sparse vectors efficiently.
34. Develop a class Matrix that represents a 2x2 matrix. Overload the ** operator to raise the matrix to a given power using matrix multiplication.
35. Design a class Polynomial representing a polynomial. Overload the // operator to compute the derivative of the polynomial.
36. Design a class Currency that represents a specific amount of money in a given currency (e.g., USD, EUR). Overload the + and - operators to allow operations between instances of the same currency. If an operation is attempted between different currencies, print an error message.
37. Develop a class TimeInterval that represents a period of time in hours, minutes, and seconds. Overload the % operator to compute the remainder of dividing two time intervals.
38. Construct a class Fraction that represents a fraction. Overload the ~ operator to compute the reciprocal of the fraction.

STATIC METHOD

1. Design a class TemperatureConverter that has static methods to convert temperatures between Celsius and Fahrenheit.
2. Create a class StringAnalyzer that has a static method to check if a given string is a palindrome (ignores cases, spaces, and punctuation).
3. Construct a class Calculator with static methods to perform basic arithmetic operations: addition, subtraction, multiplication, and division.
4. Design a class DateValidator that has a static method to check if a given date in the format YYYY-MM-DD is valid or not.
5. Create a class NumberUtility that provides a static method to determine if a number is even or odd.
6. Design a class FileUtility that provides static methods to read and write to a file. The write method should append to a file if it exists or create a new one otherwise.
7. Construct a class Fibonacci that offers a static method to generate the Fibonacci sequence up to a given number or for a given number of terms.
8. Create a class PrimeUtility that provides a static method to determine if a number is prime and another to retrieve the nth prime number.
9. Design a class Geometry that offers static methods to calculate the area of different geometric shapes like a circle, rectangle, and triangle.
10. Develop a class DateOperations that provides static methods to determine the number of days between two dates without using any external libraries. (Assume each month has 30 days for simplicity).
11. Design a class BaseConverter that provides a static method to convert numbers from one base to another (between bases 2 and 16).
12. Construct a class BigIntegerOperations that provides a static method to compute the greatest common divisor (GCD) of two large integers represented as strings.

PROPERTY DECORTOR

1. Design a class Circle that has a private radius. Use property decorators to create getter and setter methods for the radius, ensuring the radius cannot be set to a negative value.
2. Design a class BankAccount that has a private balance attribute. Use property decorators to create a getter for the balance, ensuring that it cannot be set directly but only modified through methods like deposit and withdraw.

3. Design a class Temperature that maintains a temperature in Celsius. Use property decorators to get and set temperatures in Fahrenheit, ensuring proper conversion between Celsius and Fahrenheit.
4. Design a class Employee with private attributes: _salary and _bonus. The bonus cannot exceed 20% of the salary. Use property decorators to control access and modifications to these attributes.
5. Design a class Square that has a side attribute. Also, make properties for area and perimeter of the square. Ensure that when the side of the square is set, the area and perimeter change accordingly.
6. Design a class BankAccount where the balance cannot go below a minimum balance. If a withdrawal request would make the balance go below this limit, it should be denied.
7. Design a class Smartphone with attributes _storage_used and _total_storage. Implement properties to get available storage and storage used as percentages.

ENCAPSULATION

1. Design a class BankAccount that has a private balance. Implement methods to deposit, withdraw, and check balance. Ensure the balance never goes negative.
2. Create a class Student with private attributes for name and age. Implement public methods to get and set the name and age, ensuring age is between 5 and 25.
3. Design a class Laptop that has a private attribute for battery life (in percentage). Create methods to charge the laptop and use it, ensuring the battery percentage stays between 0 and 100.
4. Design a class SafeList that extends Python's list. This class should override the append method to ensure that only integer values can be added to the list.
5. Design a class UserProfile that has private attributes for username and password. Implement methods to get and set the username and password. Password setting should ensure that it's at least 8 characters long.
6. Develop a class Library that manages books. It should have private lists for available and issued books. Implement methods to issue, return, and check the status of a book.
7. Design a class LimitedSizeList that behaves like a list but has a maximum size. If elements are added beyond this size, the oldest elements should be removed to maintain the size limit.
8. Design a class PrivateAttributes that uses name mangling to keep all its attributes private. It should also override the default behavior of the dir() function to hide private attributes.